

Hence $\text{TreeCheck}(s)$ can be encoded as Δ_0 formulae, because it is a conjunction of the preceding five Δ_0 formulae. QED

Lemma A.3. In this paper, a binary tree is defined as a rooted tree, where every internal node has *one or two* children. There exists a Δ_0 formula, $\text{BinaryTreeCheck}(s)$, which indicates whether or not a string s so encodes a binary tree.

Proof. Equation (A.4) is a Δ_0 formula which yields FALSE when s is a binary tree, yields TRUE when s is a non-binary tree, and its truth-value is unpredictable when s is not a tree.

$$\begin{aligned} \exists x_1 < \text{Length}(s) \exists x_2 < x_1 \exists x_3 < x_2 \exists x_4 < x_3 : & \text{TreeCheck}(\text{Substring}(s, x_2, x_1)) \wedge \\ & \text{TreeCheck}(\text{Substring}(s, x_3, x_2 - 1)) \wedge \text{TreeCheck}(\text{Substring}(s, x_4, x_3 - 1)) \end{aligned} \quad (\text{A.4})$$

Since both $\text{TreeCheck}(s)$ and Equation (A.4) have Δ_0 encodings, so will there be a Δ_0 encoding for the statement that $\text{TreeCheck}(s)$ is true and Equation (A.4) is false. The latter formula is a Δ_0 encoding for $\text{BinaryTreeCheck}(s)$. QED

Let s denote a byte string encoding a tree with N nodes in parenthesized notation. Each node v of this tree will be associated with a unique principle string. Define the index of v to be the integer indicating the number of "{" symbols which are positioned in the byte string s to the left of v 's principle string. (Each of the N nodes in a byte string s will be thus associated with an unique index number between 1 and N .) Lemma A.4 (below) describes functions, ranging over the set of index numbers, induced by s .

Lemma A.4. Five further compound functions, which are well defined whenever s is a parenthesized encoding of a tree, are:

- a) $\text{NODE}(i, s)$ designating the byte address of the i -th symbol "{" in s if this symbol exists (and zero otherwise).
- b) $\text{INDEX}(j, s)$ indicating how many repetitions of the symbol "{" appear among the first j bytes of s . (Since $\text{INDEX}(\text{NODE}(i, s), s) = i$, the INDEX function can be viewed intuitively as the inverse of the NODE function.)
- c) $\text{PRINCIPLE}(i, s)$ representing the substring which is $\text{NODE}(i, s)$'s principle string.
- d) $\text{ANCESTOR}(i, L, s)$ indicating the index of the ancestor of i which lies L levels above i , if such an ancestor exists (and where $\text{ANCESTOR}(i, L, s) = 0$ if this ancestor does not exist in s);
- e) $\text{PARENT}(i, s) = \text{ANCESTOR}(i, 1, s)$

Proof The formal encodings of functions (a) thru (d) appear below:

$$\text{NODE}(i, s) = \text{Find}(s, i, \text{"{"}) \quad (\text{A.5})$$

$$\text{INDEX}(j, s) = \text{Count}(\text{Substring}(s, 1, j), \text{"{"}) \quad (\text{A.6})$$

$$\text{PRINCIPLE}(i, s) = \text{Substring}(s, \text{NODE}(i, s) + 1, \text{Bracket}(s, \text{NODE}(i, s)) - 1) \quad (\text{A.7})$$

$$\text{ANCESTOR}(i, L, s) = \text{INDEX}(\text{ReverseAddress}(\text{Substring}(s, 1, \text{NODE}(i, s)), \text{"{"}, \text{"}"}, L + 1), s) \quad (\text{A.8})$$

Notation: The exponent λ will always denote a fixed constant employed to establish the range for the quantifiers $\forall v < x^\lambda$ and $\exists v < x^\lambda$. We will demonstrate how various logical constructs can be expressed with formulae so "majorized" by λ . Lemma 5.1 indicated that each "majorized formula" can be transformed into a Δ_0 formula. Thus while the next fourteen propositions technically discuss only majorized formulae, they implicitly indicate the existence of Δ_0 formulae as well.

Lemma A.5. Let $\text{FormulaCheck}(x)$ express that the byte string x is a well defined encoding of a formula. Then $\text{FormulaCheck}(x)$ can be encoded as a majorized formula.

Proof Sketch. Since the set of well-defined formulae is a context free language [ASU86, LRS76], each such formula x must be associated with some string $t < x^\lambda$ such that t is a parse tree confirming that x is a well defined encoding. Let $\text{PARSE}(t, x)$ therefore represent the formula indicating that t is a parse tree showing that x is well defined. Without loss of generality, let us assume that the principle string of the i -th leaf in the tree t consists of a 2-byte sequence "iB", where "i" is a special punctuation mark indicating the presence of a leaf, and B is a byte corresponding to the i -th byte of x . It is easy to prove that one can formally encode the relation $\text{PARSE}(t, x)$ as a Δ_0 formula. This is because

- i) For each individual production P in the Grammar, one can trivially encode a Δ_0 formula $\text{Test}_P(i, t)$, which expresses that the i -th node in the string t is a root node for the production P . For example, if P represents $E \rightarrow FG$, then $\text{Test}_P(i, t)$ would be the Δ_0 formula

$$\exists j < t \exists k < t \forall m < t : j < k \wedge \{ \text{PRINCIPLE}(i, t) = E \wedge \text{PRINCIPLE}(j, t) = F \wedge \quad (\text{A.9})$$

$$\text{PRINCIPLE}(k, t) = G \wedge \text{PARENT}(j, t) = i = \text{PARENT}(k, t) \wedge (\text{PARENT}(m, t) = i \supset m = j \text{ or } k) \}$$

Thus for any context free grammar with L productions, one can construct a Δ_0 formula $\text{Test}_1(i, t) \vee \text{Test}_2(i, t) \vee \dots \vee \text{Test}_L(i, t)$ indicating that the i -th node satisfies one of the Grammar's L productions. Then (A.10) is a Δ_0 formula asserting that t is a valid parse

tree. (In Eq (A.10), the parenthesis expression to the left of $\supset$ indicates that i is a valid index number, and it is presumed that the Grammar's terminal symbols [ASU86,LRS76] are recognized by a degenerate production $\text{Test}_0(i,t)$, which has an empty string to the right of " $\rightarrow$ ".)

$$\forall i < t : \text{NODE}(i, t) \neq 0 \supset \{ \text{Test}_0(i,t) \vee \text{Test}_1(i,t) \vee \text{Test}_2(i,t) \vee \dots \vee \text{Test}_L(i,t) \} \quad (\text{A.10})$$

- ii) Equation (A.11) is a Δ_0 formula which states that $\forall i \leq \text{Length}(x)$ if B is the i -th byte of x then the i -th leaf of t will store " B " in its principle string.

$$\forall i \leq \text{Length}(x) : \text{Byte}(x, i) = \text{Byte}(t, \text{Find}(t, i, "!") + 1) \quad (\text{A.11})$$

The formal Δ_0 encoding of $\text{PARSE}(t,x)$ will thus be the assertion that the conjunction of $\text{TreeCheck}(t)$ and Equations (A.10) and (A.11) holds.

The first sentence of this proof indicated that x is an encoding of a formula *if and only if* $\exists t < x^\lambda$ satisfying $\text{PARSE}(t,x)$. Since $\text{PARSE}(t,x)$ was just shown to have a Δ_0 encoding, it follows that $\text{FormulaCheck}(x)$ has a majorized encoding. QED

Lemma A.6 There exists a majorized formula, $\text{SentenceCheck}(x)$, indicating that x is a well defined encoding of a sentence.

Proof. Without loss of generality we may assume that a formula's every universal quantifier declaration " $\forall \hat{v}[k]$ " and every existential quantifier declaration " $\exists \hat{v}[k]$ " is immediately followed by a pair of left and right parenthesis symbols indicating their range. The left and right parenthesis symbols "(" and ")" within a formula clearly induce an implicit tree structure, analogous to the curly brackets "{" and "}" providing a tree structure for the encoding of a proof. Thus using an analog of Lemma A.4's ANCESTOR function, we can easily devise a Δ_0 formula $\text{Test}_1(i,j,k,x)$ which yields True when $i < j \wedge \text{Byte}[x,i] = \text{Byte}[x,j] = "\hat{v}"$ and $\text{Byte}(x,j)$ is the initial byte of a substring " $\hat{v}[k]$ " that lies within the range of some quantifier declaration, " $\forall \hat{v}[k]$ " or " $\exists \hat{v}[k]$ ", indicated by x 's i -th byte.

Also, it is easy to develop a Δ_0 formula $\text{Test}_2(j)$ which yields True when $\text{Byte}[x,j] = "\hat{v}"$, and it corresponds to the second byte of a substring " $\forall \hat{v}[k]$ " or " $\exists \hat{v}[k]$ ", lying within x . Since $\text{Test}_1(i,j,k,x)$ and $\text{Test}_2(j)$ have Δ_0 definitions, so can (A.12) be expressed as a Δ_0 formula.

$$\forall j \leq \text{Length}(x) \exists i \leq j \exists k < x : \text{Byte}(x, j) = "\hat{v}" \supset \text{Test}_1(i, j, k, x) \vee \text{Test}_2(j) \quad (\text{A.12})$$

It is clear that a string x represents a sentence if and only if it satisfies the conjunction of $\text{FormulaCheck}(x)$ and Equation (A.12). Since both these formulae have majorized encodings, so

must also the formula $\text{SentenceCheck}(x)$, asserting their conjunction, be majorized. QED

Lemma A.7 There exists a majorized formula $\text{ReplaceCheck}(x_1, s_1, x_2, s_2)$ which specifies that x_1 and x_2 are identical byte strings, except that appearances of the substring s_1 within x_1 are replaced by s_2 in x_2 .

Proof. There are many different simple methods for encoding $\text{ReplaceCheck}(x_1, s_1, x_2, s_2)$ as a majorized formula. We will illustrate one.

Say two byte strings d and e double encode the strings x_1 and x_2 iff e enumerates in sequential order the bytes from x_1 and x_2 subject to the constraint that $\text{Byte}(e, j)$ represents an element of the string x_i precisely when $\text{Byte}(d, j) = i$ or 3. For $i = 1$ or 2, this condition will be encoded by the Δ_0 formula $\text{Test}_i(d, e, x_i)$ indicated below:

$$\begin{aligned} \forall m \leq \text{Length}(x_i) \exists n \leq \text{Length}(e) : \{ 0 < m = \text{Count}(\text{Substring}(d, 1, n), 3) + \text{Count}(\text{Substring}(d, 1, n), i) \wedge \text{Byte}(d, n) = i \text{ or } 3 \} \\ \supset \text{Byte}(e, n) = \text{Byte}(x_i, m) \end{aligned} \quad (\text{A.13})$$

An analogous Δ_0 formula can enforce the constraint that $\text{Byte}(d, j)$ always equals 3 except when

- i) d contains a block of consecutive “1” bytes of cardinality = $\text{Length}(s_1)$ followed by a block of consecutive “2” bytes of cardinality = $\text{Length}(s_2)$, and
- ii) the corresponding fragment of e enumerates the substring s_1 followed by s_2 .

Hence there exists a Δ_0 formula, denoted as $\text{ReplaceTest}(x_1, s_1, x_2, s_2, d, e)$, which enforces constraints (i), (ii) and $\text{Test}_1(d, e, x_1) \wedge \text{Test}_2(d, e, x_2)$ simultaneously. Then a majorized formula expressing the condition $\text{ReplaceCheck}(x_1, s_1, x_2, s_2)$ will be

$$\exists d < \{\text{Max}(x_1, x_2)\}^3 \exists e < \{\text{Max}(x_1, x_2)\}^3 : \text{ReplaceTest}(x_1, s_1, x_2, s_2, d, e) \quad (\text{A.14})$$

QED

Observation A.8. Several of the proofs in this Appendix will require that we be quite precise about how an integer will be encoded as a bit string. Two methods for encoding integers will be employed, its natural encoding and its godelized encoding. The former representation of an integer i will be denoted as simply i : it will denote i 's representation as a base-2 number. The godelized encoding of i , defined in Section 5, represented i as a sequence of $\lfloor \log_{32} i \rfloor + 3$ bytes, including an initialization byte B_1 , a termination byte B_2 , and $\lfloor \log_{32} i \rfloor + 1$ main bytes for encoding i as a base-32 number. (Without loss of generality we may assume that a base-32 digit b will be formally represented by the 6-bit code $32+b$.) Henceforth i^* will denote i 's godelized

encoding. Since $i^* > i^{6/5}$, $IS(A)$ will be incapable of recognizing that the mapping of i onto i^* constitutes a total function. However, it will be able to recognize *as a relation*, the Δ_0 formula in Equation (A.15), denoted as $ConvertCheck(i, i^*)$, which expresses this relationship between i and i^* . (The formal encoding of $ConvertCheck(i, i^*)$, in Equation (A.15), utilizes the fact that multiplication by the *scalar constants* of 5 and 6 is recognized by $IS(A)$ as total functions.)

$$\forall m < Length(i^*) \forall n \leq 5 : (m \geq 2 \wedge n > 0) \supset Bit(i^*, 6m) = 1 \wedge Bit(i^*, n+6(m-1)) = Bit(i, n+5(m-2)) \wedge \\ Byte(i^*, 1) = B_1 \wedge Byte(i^*, Length(i^*)) = B_2 \quad (A.15)$$

Corollary A.9. Part (a) of Theorem A.1 is a consequence of the preceding lemmas. (That is, $SUBST_i(x, y)$ can be encoded as a majorized formula.)

Proof. Let $Test_i(x)$ be a formula which yields TRUE iff x is the Godel number of a formula which contains at least one appearance of the variable $\hat{v}[i]$, and every occurrence of $\hat{v}[i]$ in x is free. Lemma A.5 trivially implies $Test_i(x)$ can be encoded as a majorized formula. Since $ReplaceCheck$ and $ConvertCheck$ also have majorized encodings, it is apparent that Equation (A.16) (below) is a majorized encoding of $SUBST_i(x, y)$.

$$\{ \exists s \leq y : ConvertCheck(x, s) \wedge ReplaceCheck(x, y, \lceil \hat{v}[i] \rceil, s) \wedge Test_i(x) \} \vee \{ \neg Test_i(x) \wedge y = 0 \} \quad (A.16)$$

QED

Observation A.10. Define the formula $ConcatenateCheck_m(z, x_1, x_2, \dots, x_m)$ to indicate that the byte string z is begotten by concatenating $x_1, x_2, \dots, x_m$. While $IS(A)$ is too weak to prove that concatenation is a total function over the domain arguments $x_1, x_2, \dots, x_m$, it certainly can represent $ConcatenateCheck_m(z, x_1, x_2, \dots, x_m)$ as a Δ_0 formula. For instance, Equation (A.17) is a Δ_0 representation of $ConcatenateCheck_2(z, x_1, x_2)$.

$$Substring(z, 1, Length(x_1)) = x_1 \wedge Substring(z, Length(x_1)+1, Length(z)) = x_2 \quad (A.17)$$

Lemma A.11. There exists a majorized formula $InductAx(s)$ which indicates that s is a Godel number of one of Peano Arithmetic's induction axioms.

Proof. Peano Arithmetic's induction axioms consists of all sentences of the form:

$$\Phi(0) \wedge \forall v (\Phi(v) \supset \Phi(v+1)) \supset \forall w \Phi(w) \quad (A.18)$$

Define the focal substrings of Equation (A.18) to be its subformulae " $\Phi(0)$ ", " $\Phi(v)$ ", " $\Phi(v+1)$ " and " $\Phi(w)$ ". Since as byte strings these subformulae are identical except for a change in their base term, Lemma A.7 implies a majorized formula can recognize when two strings are identical except for

such a modification. Hence, there shall exist a majorized formula $\text{Test1}(\alpha_1, \alpha_2, \alpha_3, \alpha_4)$ that checks whether four strings $\alpha_1, \alpha_2, \alpha_3$ and α_4 differ only by replacing "0" in the first string with "v", "v+1" and "w" in the latter three strings.

Define $\text{Test2}(s, \alpha_1, \alpha_2, \alpha_3, \alpha_4)$ to be the statement that the string s is begotten by concatenating the four focal substrings $\alpha_1 \equiv \Phi(0)$, $\alpha_2 \equiv \Phi(v)$, $\alpha_3 \equiv \Phi(v+1)$, and $\alpha_4 \equiv \Phi(w)$ in the manner indicated by Equation (A.18). By Observation A.10, there will exist a Δ_0 encoding of $\text{Test2}(s, \alpha_1, \alpha_2, \alpha_3, \alpha_4)$.

Also, Lemma A.6 indicates that the formula $\text{SentenceCheck}(s)$ is a majorized expression that verifies that s is indeed a well defined sentence. The preceding observations imply that a majorized encoding for $\text{InductAx}(s)$ is

$$\exists \alpha_1 < s \exists \alpha_2 < s \exists \alpha_3 < s \exists \alpha_4 < s : \text{Test1}(\alpha_1, \alpha_2, \alpha_3, \alpha_4) \wedge \text{Test2}(s, \alpha_1, \alpha_2, \alpha_3, \alpha_4) \wedge \text{SentenceCheck}(s) \quad (\text{A.19})$$

Corollary A.12. The three assertions of Part (b) of Theorem A.1 are valid. *That is, there exist three majorized formulae which confirm whether a Godel number is an axiom of Peano Arithmetic (PA), of PA+, or of Zermello Fraenkel Set Theory.*

Proof. Lemma A.11 showed that the induction axioms of PA can be recognized by a majorized formula. Since the remaining noninduction axioms of Peano Arithmetic *are only finite in number*, a majorized formula must certainly be able to also recognize them. Hence, Peano Arithmetic certainly satisfies Corollary A.12.

Similar proof methodologies apply also to PA+ and Zermello Fraenkel Set Theory, since they contain also only a finite number of axioms outside an easily recognized infinite schema. **QED**

Lemma A.13 The notion of a "Positive Rewrite" was defined in Section 5. *There exists a majorized formula $\text{PositiveCheck}(g, h)$ which checks whether h equals the Positive Rewrite of g .*

Proof Outline For the sake of simplifying the proof's notation, we will assume that each sentence g is written in a syntax where its first "(" symbol appears to the immediate right of its rightmost broad quantifier. Define $\text{Prefix}(g)$ to be the fragment of byte string g that lies to the immediate left of this "(" symbol, and define $\text{Suffix}(g)$ to be the remainder of g that lies to the right of $\text{Prefix}(g)$. Both these functions are recognized by $\text{IS}(A)$, since they can be encoded by combining the functions Substring , Find and Length , as illustrated below:

$$\text{Prefix}(g) = \text{Substring}(g, 1, \text{Find}(g, 1, "(") - 1) \quad (\text{A.22})$$

$$\text{Suffix}(g) = \text{Substring}(g, \text{Length}(\text{Prefix}(g)) + 1, \text{Length}(g)) \quad (\text{A.23})$$

Also define $\text{Parity}(g,i)$ to be a function that equals zero iff there are an even number of appearances of the symbol " $\neg$ " among the first i bytes of g , and one iff there are an odd number of appearances of " $\neg$ ". Equation (A.24) illustrates how $\text{Parity}(g,i)$ can be encoded.

$$\text{Parity}(g) = \text{And}(1, \text{Count}(\text{Substring}(g, 1, i), "\neg")) \quad (\text{A.24})$$

Define $\text{Test1}(g,x)$ to be a formula that is satisfied only by the unique string x which is identical to $\text{Prefix}(g)$, except that its i -th byte is changed from " $\forall$ " to " $\exists$ " (and vice versa) when $\text{Parity}(g,i) = 1$. A Δ_0 encoding of the formula $\text{Test1}(g,x)$ is given below:

$$\begin{aligned} \forall i \leq \text{Length}(\text{Prefix}(g)) : \{ & [(\text{Parity}(g,i)=0 \vee \text{Byte}(g,i) \neq "\exists" \vee \text{Byte}(g,i) \neq "\forall") \supset \text{Byte}(x,i) = \text{Byte}(g,i)] \wedge \\ & [(\text{Parity}(g,i)=1 \wedge \text{Byte}(g,i) = "\exists") \supset \text{Byte}(x,i) = "\forall"] \wedge \\ & [(\text{Parity}(g,i)=1 \wedge \text{Byte}(g,i) = "\forall") \supset \text{Byte}(x,i) = "\exists"] \} \\ & \wedge \text{Length}(x) = \text{Length}(\text{Prefix}(g)) \end{aligned} \quad (\text{A.25})$$

Define $\text{Test2}(x,y)$ to be a formula which asserts that x and y are identical byte strings except that each appearance of the symbol " $\neg$ " in x is removed from y . Lemma A.7 implies that $\text{Test2}(x,y)$ will have a majorized encoding.

Define $\text{Test3}(g,h,x,y)$ to be a formula which asserts that $\text{Test1}(g,x)$ and $\text{Test2}(x,y)$ are true and that h equals

- i) the concatenation of y with $\text{Suffix}(g)$ when $\text{Parity}(g, \text{Length}(\text{Prefix}(g))) = 0$;
- ii) the triple concatenation of y , the symbol " $\neg$ " and $\text{Suffix}(g)$ when $\text{Parity}(g, \text{Length}(\text{Prefix}(g))) \neq 0$.

The combination of the preceding discussion and Observation A.10 imply that $\text{Test3}(g,h,x,y)$ can be provided with a majorized encoding. Hence, a majorized formula representing $\text{PositiveCheck}(g,h)$ will be $\exists x < g^2 \exists y < g^2 : \text{Test3}(g, h, x, y)$ QED.

We are now almost ready to prove Part (c) of Theorem A.1. Define a sentence tree to be a binary tree where the principle string of each node is a well-defined sentence. Equation (A.26), henceforth called $\text{SentenceTreeCheck}(t)$, will verify that a byte string t is a sentence tree. Since Lemmas A.3 and A.6 indicated that $\text{BinaryTreeCheck}(t)$ and $\text{SentenceCheck}(s)$ are majorized formulae, it follows that Equation (A.26) represents $\text{SentenceTreeCheck}(s)$ as a majorized formula.

$$\text{BinaryTreeCheck}(t) \wedge \{ \forall i < t : \text{NODE}(i,t) \neq 0 \supset \text{SentenceCheck}(\text{PRINCIPLE}(i,t)) \} \quad (\text{A.26})$$

The following lemma is the main component of the proof of Part (c) of Theorem A.1.

Lemma A.14. Assume that the byte string t satisfies $\text{SentenceTreeCheck}(t)$. Then there exists a majorized formula $\text{DeductionCheck}(i,t)$ which specifies that the sentence $\text{PRINCIPLE}(i,t)$ is a valid deduction from some ancestor of $\text{NODE}(i,t)$, using the standard semantic tableaux deduction rules [Sm68,Fi90] for eliminating the symbols “ $\wedge$ ”, “ $\vee$ ”, “ $\supset$ ”, “ $\forall$ ”, “ $\exists$ ”, and right shifting and/or eliminating “ $\neg$ ”.

Proof. The twelve permissible deduction rules in a semantic tableaux proof are listed in Figure 1. A rule of the form $\alpha \rightarrow \beta$ indicates that β is a valid deduction from its ancestor α . Also the rule $\alpha \rightarrow \beta \mid \gamma$ in Figure 1 indicates that the two *sibling* nodes β and γ may be *jointly inferred* from their common ancestor α .

For $j \leq 9$ and arbitrary sentences α and β (encoded as byte strings), let $\text{Test}_j(\alpha, \beta)$ be a formula which is satisfied when the concatenated byte string “ $\alpha \rightarrow \beta$ ” satisfies Rule j in Figure 1. For $j \leq 8$, each $\text{Test}_j(\alpha, \beta)$ can be encoded as a majorized formula by employing ConcatenateCheck in a straightforward manner. The formula $\text{Test}_9(\alpha, \beta)$ for Rule 9 can likewise be encoded very simply by using ReplaceCheck .

Let $\text{Check}_j(i,t)$ be a formula stating that $\text{PRINCIPLE}(i,t)$ is a deduction in the proof tree t by applying the Rule j to one of the ancestors of $\text{NODE}(i,t)$. For $j \leq 9$, Equation (A.27) is a majorized formula expressing this relationship.

$$\exists L \leq t : \text{Test}_j(\text{PRINCIPLE}(\text{ANCESTOR}(i,L,t), t), \text{PRINCIPLE}(i,t)) \quad (\text{A.27})$$

For $j = 10, 11$ or 12 , an only slightly more complicated construction is needed to formulate $\text{Check}_j(i,t)$ as a majorized formula. The only further requirement to encode Rule 10 is that the parameter u must not be also employed by any of $P(u)$'s ancestors; this condition can be easily encoded as a majorized formula using the ANCESTOR and PRINCIPLE functions from Lemma A.4. The added requirement for encoding $\text{Check}_{11}(i,t)$ and $\text{Check}_{12}(i,t)$ is that the *sibling of* $\text{NODE}(i,t)$ must satisfy the special constraints required by Rules 11 and 12. This is also easily encoded as a majorized formula using the PARENT and PRINCIPLE functions from Lemma A.4.

Since for each $j \leq 12$, $\text{Check}_j(i,t)$, can be encoded as a majorized formula, the formula $\text{Check}_1(i,t) \vee \text{Check}_2(i,t) \vee \cdots \vee \text{Check}_{12}(i,t)$ will be a majorized representation for $\text{DeductionCheck}(i,t)$. QED

Corollary A.15. Part (c) of Theorem A.1 is valid. *That is, for any axiom system B which can be encoded by a majorized formula, $\text{Prf}_B(x, y)$ can also be encoded as a majorized formula.*

Proof. A byte string y will be a semantic tableaux proof of x from an axiom system B iff it is a sentence tree satisfying the following *three* constraints:

- I) Its root is the positive rewrite of the negation of the sentence with Godel number x .
- II) Every other sentence in the tree y will either be an axiom of B or a deduction from a higher tree node (via Lemma A.14's DeductionCheck formula).
- III) Every branch in the proof tree y is closed.

To prove Corollary A.15, we must show that there exists majorized formulae for enforcing the three constraints above. The construction of such formulae are given below:

- 1) Let $\text{Negation}(x, g)$ be a Δ_0 formula which asserts that g is a byte string identical to x except that g contains one additional byte " $\neg$ " as its lead symbol. Let $\text{NR}(x, h)$ be the formula

$$\exists g \leq x^2 : \text{Negation}(x, g) \wedge \text{PositiveCheck}(g, h) \quad (\text{A.28})$$

Since PositiveCheck was shown by Lemma A.13 to have a majorized encoding, the preceding formula also has a majorized encoding. Hence constraint I can also be checked by the majorized formula $\text{NR}(x, \text{PRINCIPLE}(1, y))$.

- 2) The hypothesis of Corollary A.15 implies there exists a majorized formula, henceforth denoted as $\text{AxiomCheck}(i, y)$, which indicates whether or not $\text{PRINCIPLE}(i, y)$ is an axiom of B . Since Lemma A.14 indicated that $\text{DeductionCheck}(i, y)$ was another majorized formula, it follows that Equation (A.29) is a third majorized formula, which checks that the proof tree y satisfies constraint II.

$$\forall i < y : 0 \neq \text{NODE}(i, y) \neq 1 \supset \text{DeductionCheck}(i, y) \vee \text{AxiomCheck}(i, y) \quad (\text{A.29})$$

- 3) Let $\text{Leaf}(i, y)$ denote a formula which verifies that the i -th node of the tree y is a leaf, and $\text{Closed}(i, y)$ be a formula which indicates that the tree branch associated with i is closed (i.e. that it contains both some sentence Φ and its negation $\neg\Phi$.) Both these relations can be trivially encoded as majorized formulae, using Lemma A.4's ancestor function and the $\text{Negation}(x, g)$ relation. A majorized formula enforcing constraint III would then simply be:

$$\forall i < y : \text{Leaf}(i, y) \supset \text{Closed}(i, y). \quad (\text{A.30})$$

QED

The next two lemmas will help prove Part (d) of Theorem A.1

Lemma A.16 There exists a majorized formula $AxGr_1(x)$ which expresses that x is the Godel number of one of $IS(A)$'s Group-1 axioms.

Proof Sketch. The set G_0 of $IS(A)$'s Group 1 axioms was defined only informally in Section 2. It will now be necessary for us to define these axioms more formally in section 5's notation. For each integer $k > 1$, G_0 will include two axioms of the precise syntactical form:

$$\hat{c}[k] = \hat{c}[k-1] + \hat{c}[1] \quad (A.31)$$

$$\forall v (v < \hat{c}[k] \supset (v = \hat{c}[0] \vee v = \hat{c}[1] \vee v = \hat{c}[2] \vee \cdots \vee v = \hat{c}[k-1])) \quad (A.32)$$

Let $Test1(x)$ and $Test2(x)$ be two formulae that check whether a Godel number x is a sentence of the respective forms (A.31) and (A.32). Both these tests can be encoded as majorized formulae. $Test1(x)$ is encoded by using `ConvertCheck` and `ConcatenateCheck` in a very straightforward manner. The majorized encoding of $Test2(x)$ will essentially have two parts. Its first fragment will employ the function `Substring` and the formula `ConvertCheck` to verify that the initial bytes in x correspond to the string " $\forall v (v < \hat{c}[k] \supset (v = \hat{c}[0] \vee$ ", and its final sequence of bytes consists of the string " $v = \hat{c}[k-1]))$ ". The second fragment of $Test2$ will verify that for each $j < k-2$, the byte string " $v = \hat{c}[j] \vee$ " (in the middle section of x) is immediately followed by string the " $v = \hat{c}[j+1]$ " (by employing the primitives `Substring` and `ConvertCheck` again in a straightforward manner.)

The axiom group G_0 was defined so that it would contain *only a finite number of axioms* beyond the two infinite schemes of the form (A.31) and (A.32). Hence since the majorized formulae $Test1(x)$ and $Test2(x)$ can determine whether x lies in one of these two classes, it is also possible to provide the full formula $AxGr_1(x)$ with a majorized encoding. QED

Lemma A.17. For each nice axiom system A , there exists a majorized formula $AxGr_2(x)$ which expresses that x is the Godel number of one of $IS(A)$'s Group-2 axioms.

Proof. For each Π_1 sentence Φ , Equation (A.33) will represent the corresponding Group-2 axiom of $IS(A)$:

$$\forall v (Prf_A(\ulcorner \Phi \urcorner, v) \supset \Phi) \quad (A.33)$$

It is easy to construct a majorized formula for recognizing the Godel number of a Group-2 axiom, by combining the results from Lemmas A.6, A.8, A.10 and A.15. In particular, it is trivial to extend Lemma A.6 to construct a majorized formula $Test(s)$ which recognizes when a byte string s

formally corresponds to the encoding of some Π_1 sentence Φ . Note that Equation (A.33) contains both such a string " Φ " and its godelized encoding " $\lceil\Phi\rceil$ ". Since $\lceil\Phi\rceil$ is the unique string satisfying $\text{ConvertCheck}(s, \lceil\Phi\rceil)$, we can use Observation A.8 to conclude that there is a Δ_0 formula for uniquely recognizing the byte string " $\lceil\Phi\rceil$ ".

Moreover, since the axiom system A is nice, the combination of Lemma 5.1 and Corollary A.15 will imply that

- i) there exists a Δ_0 formula for expressing the relation $\text{Prf}_A(\lceil\Phi\rceil, v)$, and
- ii) a majorized formula can recognize the byte string which formally encodes (i).

Finally, we observe that Equation (A.33), viewed strictly as a byte string, is begotten by concatenating the five substrings of " $\forall v("$ ", " $\text{Prf}_A(\lceil\Phi\rceil, v)$ ", " $\supset$ ", " Φ " and $"")$ ". The preceding discussion has certainly implied that each of these five component substrings of (A.33) can be individually recognized by majorized formulae. Moreover since Observation A.10 shows that arbitrary concatenations can be recognized by a Δ_0 formulae, it follows that a majorized formula can certainly determine whether or not an arbitrary string x is a Group-2 axiom, begotten by concatenating these five subcomponents. QED

Corollary A.18. Part (d) of Theorem A.1 is valid. *That is, for any nice axiom system A , the expressions $\text{Prf}_{\text{UNION}(A)}(x, y)$ and $\text{ExPrf}_{\text{UNION}(A)}(x, y, z)$ can be encoded as majorized formulae.*

Proof Let $\text{UNION}(A)$ denote the union of $\text{IS}(A)$'s Group-1 and Group-2 axioms. The combination of Lemmas A.16 and A.17 implies that there exists a majorized formula $\text{Test}(x)$, which expresses that x is an axiom of $\text{UNION}(A)$. Hence from Corollary A.15, we may conclude that another majorized formula will express $\text{Prf}_{\text{UNION}(A)}(x, y)$. Since $\text{ExPrf}_{\text{UNION}(A)}(x, y, z)$ differs from the preceding formula only by allowing the additional Godel number z as an axiom, the latter must also be expressible as a majorized formula. QED

Added Comment. As already noted, each of the theorems in this Appendix immediately translate into statements about Δ_0 formulae because Lemma 5.1 shows that every majorized formula can be translated into a Δ_0 formula. The *sole reason* that majorized formulae appeared in our discussion was that our proofs are simpler by first introducing the majorized formulae, and then relying on Lemma 5.1's translation methodology. Similarly, our discussion technically only needed G_0 's atomic functions of Addition, StringCount, Shift and Extract to encode $\text{Prf}_{\text{IS}(A)}(x, y)$ as a Δ_0 formula. The sole purpose of G_0 's functions of Andreverse, Address, Subtraction and Division (defined in Section 2) was to simplify the presentation.

APPENDIX B

Let Φ , H and Ψ have the same meanings in the proof of Proposition 6 as they did in Chapter 11, except that they now pertain to $\text{Prf}_{\text{INVALID.2}}$ rather than to $\text{Prf}_{\text{INVALID.1}}$. Thus Φ is the formula given in Equation (B.1), H is the Godel number of the sentence $\Phi(\lceil\Phi\rceil)$, and Ψ is the sentence defined by Equation (B.2).

$$\Phi \equiv \{ \forall x_2 \forall x_3 [\text{SUBST}(x_1, x_2) \supset \neg \text{Prf}_{\text{INVALID.2}}(x_2, x_3)] \} \quad (\text{B.1})$$

$$\Psi \equiv \{ \forall x \neg \text{Prf}_{\text{INVALID.2}}(H, x) \} \quad (\text{B.2})$$

To prove Proposition 6, we need to construct a semantic tableaux proof of the sentence (B.2) from the axiom system INVALID.2 , analogous to the semantic tableaux proof which had been employed in Figure 3 of Chapter 11.

The top seven levels of such a tableaux proof of Ψ are shown in Figure 4. Its first two levels have the same justification as their counterparts in Figure 3. Thus, Line 1 is justified as the temporary assumption that Ψ is false, and Line 2 is the deduction from (1) following from existential quantifier elimination (with parameter p).

The remainder of the Figure 4 is quite different from Figure 3 because INVALID.2 has no access to the $\text{L}\Pi_1$ reflection axioms. Its reflection principle is weaker, and provides only the $\text{U}\Delta_0$ reflection axioms. Let Θ_i thus denote the Δ_0 sentence:

$$\Theta_i \equiv \{ \text{SUBST}(\lceil\Phi\rceil, H) \supset \neg \text{Prf}_{\text{INVALID.2}}(H, i) \} \quad (\text{B.3})$$

Line 3 is INVALID.2 's $\text{U}\Delta_0$ reflection axiom for Θ_i . Line 4 is a deduction from (3) via universal quantifier elimination. The two sentences on Line 5 are deductions from (4) via the $\supset$ Elimination Rule. For a fixed $\lambda > 0$ whose value will be determined later, let Ω_λ and Γ denote the two Π_1 sentences defined below:

$$\Omega_\lambda \equiv \{ \forall y \forall z \exists w < z: [\text{Prf}_{\text{INVALID.2}}(H, y) \wedge z = y^\lambda] \supset \text{Prf}_{\text{INVALID.2}}(\lceil\Theta_y\rceil, w) \} \quad (\text{B.4})$$

$$\Gamma \equiv \text{SUBST}(\lceil\Phi\rceil, H) \quad (\text{B.5})$$

The two sentences at Level 6 of Figure 4 are each Group-2 Axioms of INVALID.2 . The four sentences at Level 7 are deductions from Level 6, following from the $\supset$ Elimination Rule.

Assuming the constant λ is assigned a sufficiently large initial value, the four subtrees descending from Level 7 can be all closed. This is intuitively because INVALID.2's Group-1 axiom system G_m can be shown to contradict the conjunction of the seven sentences on each branch of Figure 4's proof tree (see footnote * for a formal justification of this point). **THE MAIN POINT IS** that Chapter 11's proof of the inconsistency of INVALID.1 produces the **verbatim same** proof of the inconsistency of INVALID.2, when $\bar{q}$ from Chapter 11 is merely changed to designate Figure 4's semantic tableaux.

Summarizing Comment. The closing two paragraphs in Section 12 had explained how one would intuitively expect INVALID.2 to experience the same inconsistency problem as INVALID.1 because Lemma 12.1 showed that INVALID.2 could prove all its $L\Pi_1$ reflection sentences *as theorems* (similar to INVALID.1). *The complication in formally proving Proposition 6 was that semantic tableaux proofs allow only axioms* (and their "cut-free" deductions) to appear in the intermediate stages of a proof (i.e. provable theorems are disallowed). Therefore *unlike* Figure 3, the third line in Figure 4's proof tree *can not explicitly specify an* $L\Pi_1$ reflection sentence. Instead, Levels 4 thru 7 of Figure 4's tableaux had implicitly exploited $L\Pi_1$ reflection by constructing its essential cut-free implications. Fundamentally, this is the reason that $\bar{q}$'s construction was a bit more complicated in Proposition 6's proof than it was for Proposition 5. Since it is well known [Fi90,Ge69,Sm83,Ta87] that deductive systems with and without cuts prove the same theorems (with only a different proof length), one would intuitively expect that the only difference between the proofs of Ψ in Figures 3 & 4 to be the lengths of their proofs (as has been formally demonstrated in this Appendix).

* The leftmost branch is closed by using the fact that that when λ is assigned a sufficiently large initial value, G_m contradicts the seventh line on the leftmost branch. (In particular for such λ , $PA+$ can (trivially !!) prove Lemma 5.3's T-transformation can construct w from y and z in a manner specified by Equation (B.4), thus contradicting $\neg \text{Pr}_{PA+}(\ulcorner \Omega_\lambda \urcorner)$.) The second branch is the most revealing of the four cases because it is the only case that uses the fact that G_m recognizes multiplication as a total function. This branch is closed because G_m 's multiplication function enables it to recognize that $\forall y \exists z = y^\lambda$, and the latter invariant contradicts the combination of the second, fifth and seventh sentences on the second branch. The seventh sentence on the third branch is trivially contradicted by G_m . The second, fifth and seventh sentences on the rightmost branch contradict each other.

References

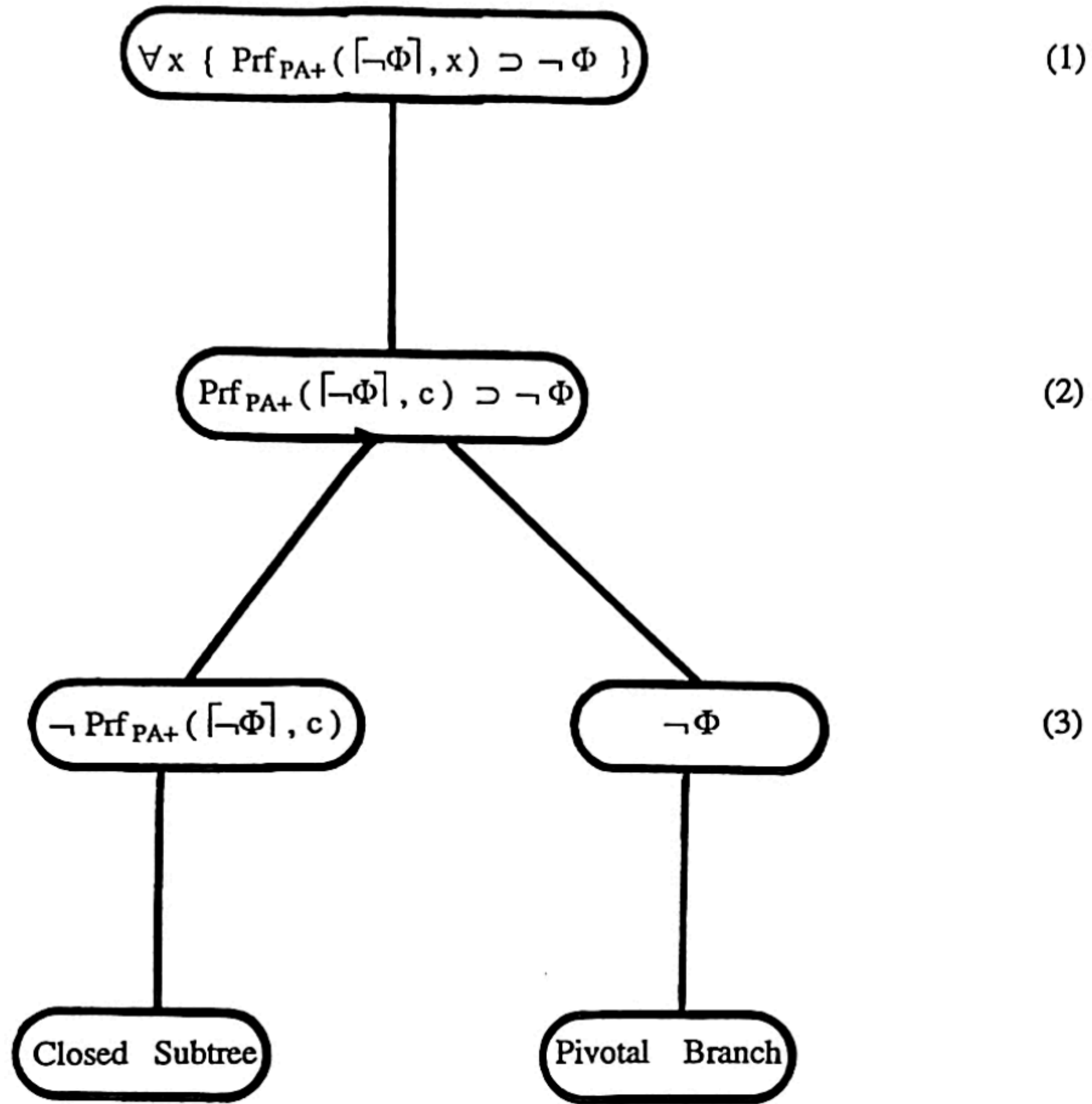
- [ASU86] Compiler Design Theory, A. Aho, R. Sethi and J. Ullman, *Compilers: Principles, Techniques and Tools*, Addison Wesley (1986).
- [Be56] E.W. Beth, Semantical Entailment and Formal Derivability *Indag. Math.* 19 (1956) pp. 357-388.
- [Bo79] G. Boolos, *The Unprovability of Consistency: An Essay on Modal Logic* Cambridge University Press, 1979.
- [BS76] A. Bezboruah and J. Shepherdson, Godel's Second Incompleteness Theorem for Q, *Journal of Symbolic Logic* 41 (1976) pp. 503-512.
- [Bu85] S. Buss, Polynomial Hierarchy and Fragments of Bounded, *Proceedings of 17th Annual ACM Symposium on Theory of Computing* (1985) pp. 285-290
- [Bu86] S. Buss, *Bounded Arithmetic*, Princeton Ph. D. dissertation published in Proof Theory Lecture Notes #3 by Bibliopolis (1986), see also [Bu85].
- [En72] H. Enderton *A Mathematical Introduction to Logic* Academic Press 1972.
- [Fe60] S. Feferman, Arithmetization of Metamathematics in a General Setting, *Fundamenta Mathematicae* 49 (1960) pp. 35-92.
- [Fe62] S. Feferman, Transfinite Recursive Progressions of Axiomatic Theories, *J. of Symbolic Logic*, 27 (1962) 259-316.
- [Fe88] S. Feferman, Finitary Inductively Presented Logics, in *Proceedings of Logic Colloquium 88*, North Holland Publishing House (1989) pp. 191-220.
- [Fi90] M. Fitting, *First Order Logic and Automated Theorem Proving*, Springer Verlag Monograph in Computer Science, 1990.
- [Ge69] G. Gentzen *Collected Papers*, translated by M.E. Szabo, North Holland, 1969.
- [Go30] K. Godel, Die Vollständigkeit der Axiome des logischen Funktionenkalküls, *Monatsh Math Phys*, 37, (1930) pp. 249-360.
- [Go31] K. Godel, Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme, I. *Monatsh Math Phys*, 38, (1931) pp. 173-198.
- [HB39] D. Hilbert and B. Bernays, *Grundlagen der Mathematik* Volume 1 (1934) and Volume 2 (1939), Springer.
- [HP87] P. Hajek and P. Pudlak, *Metamathematics of First Order Arithmetic*, Springer-Verlag (1987).
- [Je71] R. Jeroslow, Consistency Statements in Formal Mathematics, *Fundamenta Mathematicae* 72 (1971) pp. 17-40.
- [Je73] R. Jeroslow, Redundancies in the Hilbert-Bernays Derivability Conditions for Godel's Second Incompleteness Theorem *Journal of Symbolic Logic* 38 (1973) pp. 359-367.
- [Je75] R. Jeroslow, Experimental Logics and Δ_2^0 Theories, *Journal of Philosophic Logic* 4 (1975) pp. 253-267.
- [KI38] S. C. Kleene, On the Notation of Ordinal Numbers, *Journal of Symbolic Logic* 3 (1938), pp. 150-156.
- [Kr71] G. Kreisel, A Survey of Proof Theory, Part I in *Journal of Symbolic Logic* 33 (1968) pp. 321-388 (see especially footnote 8 and page 349); Part II in *Proceedings of Second Scandinavian Logic Symposium* (1971) North Holland Press (with Fenstad ed.), Amsterdam (see especially pp. 117(d) & 166).
- [KT74] G. Kreisel and G. Takeuti, Formally self-referential propositions for cut-free classical analysis and related systems, *Dissertationes Mathematicae* 118, 1974 pp. 1-55.
- [Lo55] M. Lob, Solution of a Problem of Leon Henkin, *Journal of Symbolic Logic* 20 (1955) pp. 115-118.
- [LRS76] P. Lewis, D. Rosenkrantz and R. Stearns, *Compiler Design Theory*, Addison Wesley (1976).
- [Ma74] R. Magari, Su certe teorie non enumerabili, *Ann. Mat. Pura. Appl.* 98 (1974) pp. 119-152.
- [Me87] E. Mendelson, *Introduction to Mathematical Logic*, Wadsworth & Brooks/Cole Mathematics Series, 1987.
- [Ro67] H. Rogers, *Theory of Recursive Functions and Effective Computability*, McGraw Hill 1967, see especially pp. 186-188.
- [Sc56] K. Schütte, Ein System des verknüpfenden Schliessens, *Archiv. Math. Logic Grundlagen* 2 (1956) pp. 55-67.
- [Sc83] H. Schwichtenberg, Proof Theory: Some Applications of Cut Elimination, in *Handbook on Mathematical Logic*, North Holland Publishing House (1983) pp. 867-896.
- [Sm61] R. Smullyan, *The Theory of Formal Systems*, Princeton University Press, 1961.
- [Sm68] R. Smullyan, *First Order Logic*, Springer-Verlag, 1968.
- [Sm83] C. Smoryński, The Incompleteness Theorem *Handbook on Mathematical Logic*, pp. 821-866, 1983.
- [So94] R. Solovay, private communications (April 1994).
- [St83] R. Statman, Herbrand's theorem and Gentzen's Notion of a Direct Proof, in *Handbook on Mathematical Logic*, North Holland Publishing House (1983) pp. 897-913.
- [Ta53] G. Takeuti, On a Generalized Logical Calculus, *Japan Journal on Mathematics* 23 (1953) pp. 39-96.
- [Ta87] G. Takeuti, *Proof Theory*, Studies in Logic Volume 81, North Holland, 1987.
- [Wi93] D. Willard, Self-Verifying Axiom Systems, in *Proceedings of the Third Kurt Godel Symposium* (1993) pp. 325-336, published in Springer-Verlag LNCS #713.
- [Wi94] D. Willard, The ETR Reflection Principles for Self-Verifying Axiom Systems, forthcoming report.

- 1) $P \wedge Q \rightarrow P$
- 2) $P \wedge Q \rightarrow Q$
- 3) $\neg \neg P \rightarrow P$
- 4) $\neg (P \wedge Q) \rightarrow (\neg P) \vee (\neg Q)$
- 5) $\neg (P \vee Q) \rightarrow (\neg P) \wedge (\neg Q)$
- 6) $\neg (P \supset Q) \rightarrow P \wedge \neg Q$
- 7) $\neg \forall v P(v) \rightarrow \exists v \neg P(v)$
- 8) $\neg \exists v P(v) \rightarrow \forall v \neg P(v)$
- 9) $\forall v P(v) \rightarrow P(s)$ where s is an arbitrary term
- 10) $\exists v P(v) \rightarrow P(u)$ where u is a parameter not used by any ancestor of $P(u)$
- 11) $P \vee Q \rightarrow P \mid Q$
- 12) $P \supset Q \rightarrow \neg P \mid Q$

FIGURE 1

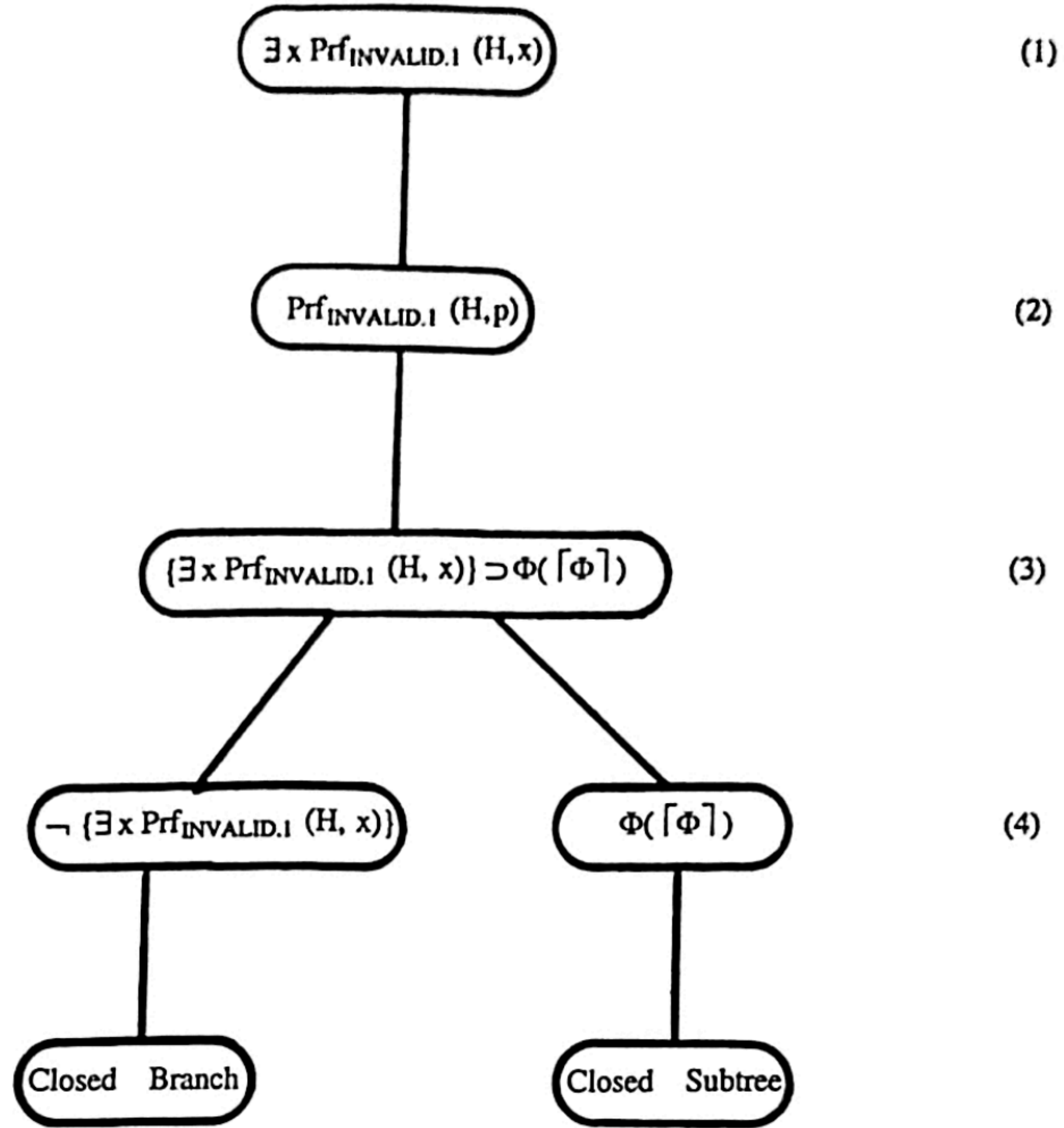
The twelve permissible deduction rules in a semantic tableaux proof are listed above. A rule of the form $\alpha \rightarrow \beta$ indicates that β is a valid deduction from its ancestor α . Also the rule $\alpha \rightarrow \beta \mid \gamma$ indicates that the two *sibling* nodes β and γ may be *jointly inferred* from their common ancestor α .

FIGURE 2



The canonical form of the top three levels of $\text{IS}(\text{PA}+)$'s terse proof of $\neg \Phi$ when Φ is a false Δ_0 sentence. Line 1 is a Group-2 Axiom of $\text{IS}(\text{PA}+)$. Line 2 is a deduction from Line 1 which introduces the constant c to eliminate the universal quantifier. The two sentences on Line 3 are deductions from (2) via the $\supset$ Elimination Rule. As noted in the proof of Lemma 7.1, the constant c is chosen to be the Godel number of $\text{PA}+$'s proof of $\neg \Phi$. Since c satisfies $\text{Prf}_{\text{PA}+}(\ulcorner \neg \Phi \urcorner, c)$, the left branch below Line 3 can be trivially closed. By definition, the presence of the sentence $\neg \Phi$ at the bottom of the "pivotal branch" implies that the proof is terse.

FIGURE 3



The top four levels of the Semantic Tableaux Proof of Ψ (stated in Equation (11.3)). Line 1 is justified as the temporary assumption that Ψ is false. Line 2 is the deduction from (1) following from existential quantifier elimination (with parameter p). Line 3 is a Π_1 local reflection axiom. The left and right sentences on Line 4 are deductions from (3) via the $\supset$ Elimination Rule. Since Line 1 contradicts the left side of line 4, this branch is closed. Also, it is easy to construct a subtree descending from $\Phi(\lceil\Phi\rceil)$ which is closed because INVALID.1's Group-1 axioms render $\Phi(\lceil\Phi\rceil)$ model theoretically incompatible with Line 2.

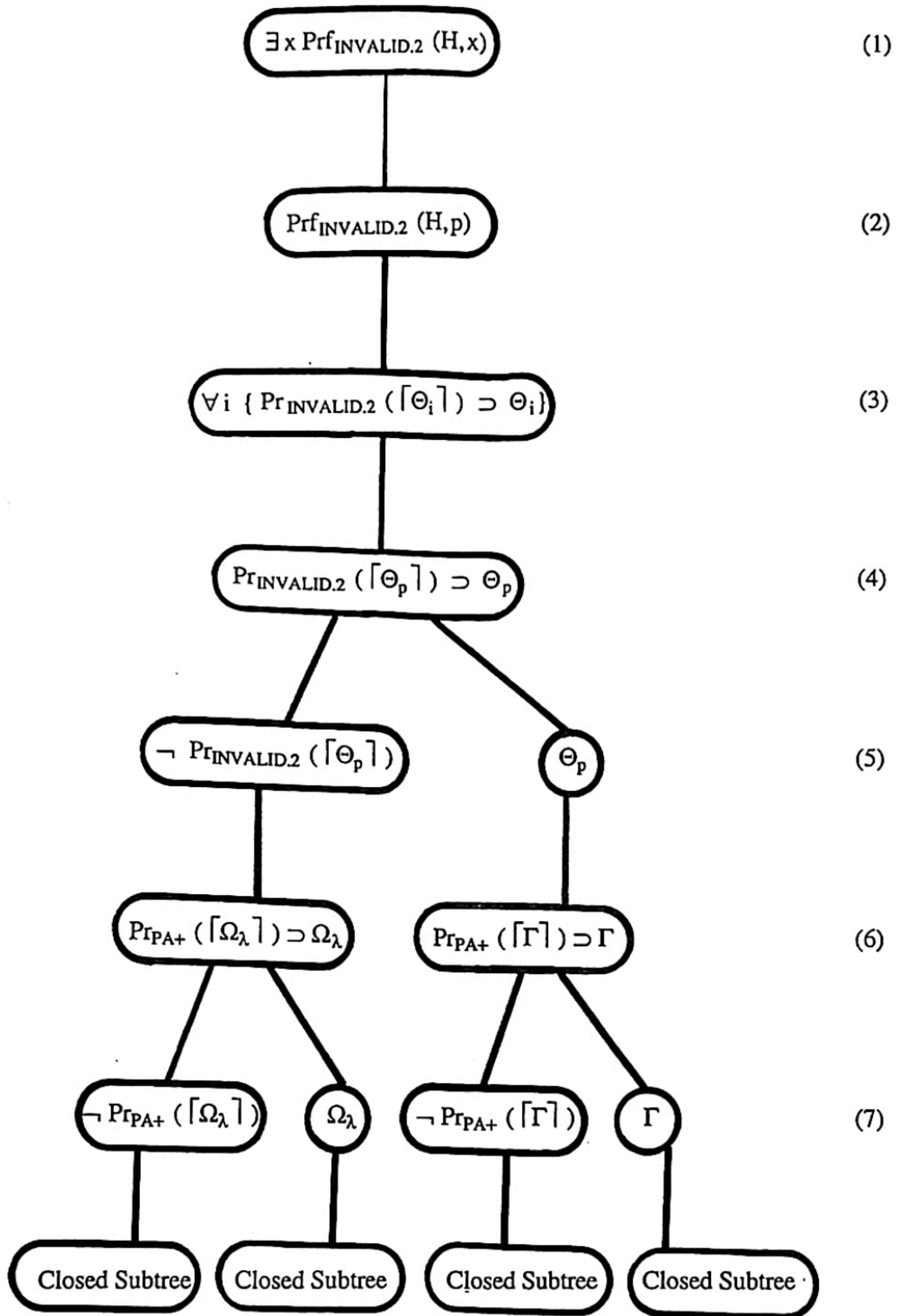


FIGURE 4. INVALID.2's semantic tableaux proof of Ψ .